

```
CREATE OR REPLACE FUNCTION bloquear_exclusao()
RETURNS TRIGGER AS
$$
BEGIN
    IF OLD.quantidade > 0 THEN
        RAISE EXCEPTION
        'Não é permitido excluir livros que ainda possuem exemplares disponíveis.';
    END IF;

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_bloquear_exclusao
BEFORE DELETE
ON livro
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();
```

```
CREATE TABLE IF NOT EXISTS log_livro (
    id_log SERIAL PRIMARY KEY,
    titulo VARCHAR(255),
    data_exclusao TIMESTAMP,
    mensagem TEXT
);
```

```
CREATE OR REPLACE FUNCTION log_exclusao_livro()
RETURNS TRIGGER AS
$$
BEGIN
    INSERT INTO log_livro (
        titulo,
        data_exclusao,
        mensagem
    )
    VALUES (
        OLD.titulo,
        NOW(),
        'Livro removido do sistema.'
    );

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_log_exclusao
AFTER DELETE
```

```
ON livro
FOR EACH ROW
EXECUTE FUNCTION log_exclusao_livro();
```

```
CREATE OR REPLACE FUNCTION validar_limite_estoque()
RETURNS TRIGGER AS
$$
BEGIN
    IF NEW.quantidade > 100 THEN
        RAISE EXCEPTION
        'Quantidade acima do limite permitido (100 unidades).';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_validar_limite
BEFORE UPDATE
ON livro
FOR EACH ROW
EXECUTE FUNCTION validar_limite_estoque();
```

a) Diferença entre BEFORE e AFTER

BEFORE é executada antes da operação acontecer no banco. Ela pode validar ou impedir a operação.

AFTER é executada depois que a operação já foi concluída com sucesso no banco.

b) Qual usar para validação?

BEFORE, porque permite verificar os dados antes da alteração e bloquear a operação quando necessário.

c) Qual usar para auditoria?

AFTER, porque o registro de auditoria deve ocorrer somente depois que a operação foi concluída com sucesso.

d) Por que a ordem é importante?

A validação deve acontecer antes da alteração para impedir dados inválidos. Já os registros de auditoria devem ocorrer depois da confirmação da operação. Isso evita inconsistências e registros incorretos nos logs.

a) Quais riscos existem ao remover todas as triggers?

Dados inválidos podem ser gravados.

Diferentes aplicações podem implementar regras de formas diferentes.

A integridade dos dados fica dependente exclusivamente do sistema.

Integrações externas podem ignorar validações.

b) Quais vantagens existem em manter regras no banco?

Centralização das regras.

Maior segurança.

Garantia de integridade dos dados.

Todas as aplicações seguem as mesmas validações.

c) Como triggers ajudam na integridade e consistência?

Triggers executam automaticamente validações e ações sempre que ocorre INSERT, UPDATE ou DELETE. Assim garantem que os dados permaneçam corretos independentemente da aplicação utilizada.

d) Exemplo real

Em sistemas bancários, triggers podem registrar automaticamente todas as movimentações financeiras em tabelas de auditoria sempre que ocorre uma transferência ou alteração de saldo.

Pra fazer os prints, cria uma tabela livro com alguns registros de teste, executa os DELETE e UPDATE mostrando a trigger funcionando e tira os prints da tela inteira. Isso costuma render todos os pontos da atividade.